State update and corrections (2026-10-22). Newer than the other documents in this file set; where they disagree, THIS wins.

Verified live facts (2026-10-22)

- Database: PostgreSQL 17.11, 15 MB used of 500 MB, 33 public tables, RLS on all; 5 migrations applied live (the repo holds more migration files because early ones were applied differently). About 42 SECURITY DEFINER functions in `public`, 28 callable by signed-in users (advisor lint 0029), none by anon. A `private` schema holds 35 helper functions (can_read_project, can_edit_project, project_role, org_role...). Extensions installed: plpgsql, supabase_vault, pg_cron, pg_stat_statements, pgcrypto (schema `extensions`), uuid-ossp; pg_trgm, unaccent, ltree, pgtap, vector available but not installed. Advisor: leaked-password protection off (Pro feature).
- Existing table layout: `docs` (body, rev, position integer, parent_ref, deleted_at...), `doc_revisions` (rev, full body snapshot, content_hash). One built-in public project readable by anon on purpose.
- Repo is now PUBLIC on GitHub (may go private again). Frontend already on Vite 8.3.4, TypeScript 7.0.2, React 18.3.1; browser, invitation function and MCP function already use `sb_publishable_` / `sb_secret_` keys (the MCP function falls back to the legacy variable). No `vercel.json` (no security headers or CSP yet). Sign-in is email + password only; no MFA; no account-creation path in the app yet. CI has no secret scanning, Dependabot, CodeQL or dependency audit.

Corrections to older documents

- The Canvas single-file build is DROPPED (owner, 2026-10-22). Ignore every statement about Canvas as a runtime.
- Supabase Free has NO managed database backups (verified on Supabase's backup page: daily backups only on Pro/Team/Enterprise; free projects must export their own data). Any statement that Free has backups is wrong.
- Supabase stops auto-granting new `public` tables AND functions to anon/authenticated/service_role: opt-out setting 2026-04-28, default for new projects 2026-05-30, all existing projects 2026-10-30 (existing objects keep their grants). Behaviour for new functions on 10-30 is not confirmed.
- Current versions (npm, 2026-10-22): react 19.3.1, zustand 5.0.16, vite 8.3.5, @codemirror/merge 6.12.3, @codemirror/lang-markdown 6.5.2, react-aria-components 1.22.1, react-router 8.5.0, @tanstack/react-router 1.170.x, jotai 3.0.1, @reduxjs/toolkit 2.13.0, rehype-sanitize 6.0.0, react-markdown 10.1.0.

Owner decisions (do not re-ask)

Zero cost (D11). English only (D2). Custom animated dropdowns stay (D13). Share links are PROJECT-WIDE, read-only, expiring, revocable (D14). Markdown source editor with preview (CodeMirror 6), not WYSIWYG.

Added after the Part C audit (2026-10-22)

- Full git history (53 commits) scanned: no secrets, no real email addresses, no zip/env/trace files; all commits authored "Claude (noreply@anthropic.com)".

- Live site headers today: Vercel already sends HSTS (`max-age=63072000; includeSubDomains; preload`); no CSP, no Referrer-Policy; `/share` returns 404; static files send `access-control-allow-origin: *`. Vercel Hobby deployment protection covers preview/deployment URLs only, not the production domain.
- Supabase Auth on Free: sessions never expire by default; time-box, inactivity timeout and single-session are Pro-only; passkeys are experimental; Turnstile/hCaptcha supported; TOTP MFA available; default SMTP is best-effort and rate limited.
- Design decisions proposed in master plan v2.1 section 3: share links at `/share#secret` with a server proxy, invite-bound account creation, TOTP + AAL2 in sensitive RPCs, real CSP after moving inline CSS.

Owner decisions added 2026-10-22 (do not re-ask)

D15 project-wide share shows all NON-restricted documents. D16 TOTP mandatory for owner, org admins and project managers. D17 the repo stays public; therefore never upload unencrypted data as workflow artifacts (any signed-in GitHub user can download artifacts; default retention 90 days). D18 Gmail will be Supabase Auth custom SMTP.

Added after the Part E+F audit (2026-10-22; details docs/22)

- Verified: Supabase pause = low activity over 7 days; restore window after a pause is 1 YEAR on the live page (the MCP docs search index still returns 90 days: stale); Edge Functions block outbound ports 25 and 587 (465 not documented), free wall-clock 150 s; Vercel Hobby cron = once a day, +-59 min; GitHub disables scheduled workflows in a public repo after 60 days without activity; Gmail app passwords need 2-Step Verification (not security-keys-only), no Advanced Protection, revoked on password change; Healthchecks Hobbyist 20 checks; UptimeRobot Free 50 monitors / 5 min; Neon Free 100 projects x 1 GB, no card; Backblaze B2 first 10 GB free; GitHub artifacts 90 days default (public max 90).
- UNVERIFIED: B2 sign-up without card (likely, phone number probably needed), personal-Gmail daily cap, `smtp.gmail.com:465`, exact auth/storage coverage of `supabase db dump`, whether a role other than `postgres` can dump RLS tables, mail-scanner behaviour.
- Design: ops_keepalive table read by Vercel cron + GitHub schedule; nightly dump restored in the same job, `age`-encrypted, 14-day artifact + B2; Healthchecks + UptimeRobot; quarterly Neon exit drill. Report code bugs listed in docs/22 section 3.
- Open owner questions: Q26 dedicated Gmail, Q27 B2 sign-up, Q28 two offline copies of the age key, Q29 Healthchecks/UptimeRobot accounts.

Product vision: a documentation workspace for many projects

The owner's goal (their words, condensed)

One app to hold the documentation of ALL their projects. Each project has **one master file** (main ideas, links/paths to phases). Each **phase** has its own files, steps, sub-steps, decisions and check marks. Nothing is fixed: chapters, sections and every document must be addable, editable, movable and deletable in the app. Each project has its own settings and functions.

Hierarchy (target)

```
Organization            (name, members, default settings, AI policy)
  └─ Project            (settings, theme, AI tokens, templates, status)
     ├─ Master file     (type: master)  one per project: vision, goals, links to every phase
     ├─ Phase           (type: phase)   order, status, owner, dates, progress %
     │  ├─ Step / Sub-step  (type: step) checkbox, status, notes, linked decisions
     │  ├─ Decision     (type: decision) question, options, choice, reason, date, status
     │  └─ Document     (type: page / chapter / spec / note / checklist / prompt ...)
     └─ Library         shared items: glossary, claims ledger, prompts, files
```

Rules: every node has a stable id (`kind:slug`), a parent, a position, a status and history (revisions). Links between nodes use ids, so renames never break links. "Progress" of a phase = done steps / total steps (computed, never typed).

What already exists (reuse, do not rebuild)

`projects` and `docs` tables (docs has ref, kind, parent_ref, title, body, rev, source native|derived), `doc_revisions` (history), AI space (change sets, Review inbox), Trash with retention, tokens with prefixes, Added pages editor (Markdown + preview + history), Realtime. The current GeminiForge spec is ONE project whose chapters are `derived` docs generated from typed tables (chapters, crates, packs...). To make it editable like everything else, migrate derived docs to `native` docs (plan below).

Build order (each step shippable and tested)

1. **Organizations**: tables `organizations`, `org_members(role owner|admin|editor|viewer)`; `projects.org_id`; RLS by membership (replace the single `app_admins` rule; keep it as super-admin). Org switcher + Project switcher in the sidebar; routes `#/o/<org>/p/<project>/...`.
2. **Project settings page**: name, description, visibility (private|unlisted|public), AI policy (propose allowed? default token expiry, default prefixes), retention days (trash), theme/accent, template.
3. **Node types + templates**: add `docs.kind` values master|phase|step|decision|checklist|page; a project template creates master + 5 phases. Step status enum todo|doing|done|blocked; decision status open|decided|superseded.
4. **Master/phase views**: master file renders a live table of phases (status, progress bar, next step); phase view lists steps with checkboxes (toggle writes a revision), decisions, linked documents. Drag-and-drop ordering (position column).
5. **Full editor for every document**: block or Markdown editor with preview, autosave drafts, revision restore, slash commands, internal links `[[kind:slug]]`, backlinks, table support (renderer exists in parts/49-

pages.tsx `Md`).

6. **Migrate the spec**: write a one-time script (scripts/) that turns each derived chapter/crate/pack/prompt row into a native doc (body Markdown + data JSON), keeps ids, then retire the sync trigger (`docs_after_write`/derived_edit path) once parity tests pass (scripts/verify-content.mjs, qa-content.mjs).
7. **Per-project AI**: tokens belong to a project (already) -> add org-level policy and per-project get_guide message (exists as `projects.ai_guide`).

Data-model sketch (SQL, to be reviewed by the owner before applying)

```
organizations(id, slug unique, name, created_by, created_at)
org_members(org_id, user_id, role, primary key(org_id,user_id))
projects add: org_id references organizations, settings jsonb default '{}', template text
docs add: position int, status text, assignee uuid, due date, data jsonb (already),
parent_ref (already)
RLS: can_read_project / can_edit_project take membership into account (functions already
centralise this: private.can_read_project, private.can_edit_project).
```

Keep ALL permission logic inside those two functions so policies and RPCs do not change.

UX principles

Chat-style reader stays for Q&A; editing is always one click away from reading ("Edit this" on any passage); every destructive action has Trash/Undo; one search box (Ctrl+K) across org > project > documents; mobile first.

Status 2026-10-22

Done: steps 1-2 of "Build order" (organizations with invited-only projects, project settings page, roles, invitation links, switcher) - see docs/08 progress log and migrations 0015*. Not done: steps 3-7 (node types and templates, master/phase/step/decision views, full editor, migration of the spec to native documents, per-project AI policy).

Architecture and folder tree (current, verified 2026-10-20)

```
geminiforge-workbench/
├─ README.md                  short intro
├─ package.json  tsconfig.json  vite.config.ts  vite.final.config.ts
├─ index.html  main.tsx       entry of the normal (Vercel) build
├─ index.final.html  main.final.tsx   entry of the single-file "Canvas" build
├─ build.mjs                  parts/*.tsx  ->  App.tsx            (git-ignored output)
├─ build-final.mjs            parts/ + canvas/ stubs -> App.final.tsx (git-ignored)
├─ prodcheck.sh               build + preview :5198 + check.mjs
│
├─ parts/                     *** SOURCE OF TRUTH of the app (28 files, joined in name
order) ***
│   ├─ 00-header.tsx          imports, icons, types (View union)
│   ├─ 10-data.tsx 11-chapters.tsx 12-content.tsx   built-in spec data and chapter text
│   ├─ 20-styles.tsx          the whole CSS string (const CSS)
│   ├─ 30-store.tsx           Zustand store, routing (parseHash/toHash/sanitizeView)
│   ├─ 31-index.tsx 35-features.tsx 36-audit.tsx 40-primitives.tsx   search index,
features, helpers
│   ├─ 45-engine.tsx 46-smart.tsx   answer engine ("ask a question")
│   ├─ 47-cloud.tsx           Supabase client, auth, cloud sync of user state
│   ├─ 48-content-live.tsx    runtime content download / version check / cache
│   ├─ 49-ai.tsx              Review inbox, AI access (tokens, message, activity)
│   ├─ 49-live.tsx            Realtime channel (owner live updates)
│   ├─ 49-pages.tsx           Added pages: Markdown renderer (Md), editor, history
│   ├─ 49-peek.tsx            side panel with highlighted passage
│   ├─ 49-trash.tsx           Trash view + ConfirmDialog/useConfirm
│   ├─ 49-tools.tsx           backup/llms.txt export, access checks, tab badge, AI trail
│   ├─ 50-composer.tsx 55-notes.tsx 60-doc.tsx 70-pane.tsx 80-views.tsx 82-settings.tsx
84-history.tsx 85-audit.tsx
│   └─ 90-app.tsx             App root, shortcuts, layout
├─ canvas/                    stubs that replace cloud parts in the Canvas build (same
file names)
│
├─ supabase/
│   ├─ migrations/            0001..0012 (+ *_down.sql). Applied live. Order matters. See
list below.
│   ├─ functions/docs-mcp/    core.mjs (tools), index.ts (Deno entry), deno.json,
test.mjs
│   └─ seed/                  seed SQL for built-in content
├─ scripts/                   content-map/make-seed-sql/seed-content/snapshot/verify-
content, mcp-e2e*.mjs
│
├─ qa-ai.mjs qa-content.mjs qa-merge.mjs qa-auto.mjs qa.mjs check.mjs ffcheck.mjs
Playwright suites
├─ dbg*.mjs prof*.mjs shots*.mjs pdf.mjs      throw-away debugging/profiling/screenshot
helpers (candidates for /tools)
└─ docs/                      all documentation (this folder)
```

Migrations (what each does)

0001 workbench_state (per-user saved state) · 0002 content (typed spec tables + RLS + get_content_bundle) · 0003 projects_docs (projects, docs, doc_revisions, MCP read functions, tokens, gate) · 0004 ai_space (change sets, drafts, risk, accept/reject) · 0005 ai_access (pause, audit view) · 0006 derived_edits (write-back to typed tables) · 0007 ai_guide · 0008 realtime · 0009 trash (soft delete, restore, purge, log) · 0010 token_edit (edit tokens, messages, versions, events) · 0011 retention (pg_cron, clear activity, audit detail, injection flag) · 0012 hardening (gate limits, policies, unicode, locks).

Data flow

Reader: browser -> `get_content_bundle` RPC (public) -> `applyContent` -> screens. Owner: browser (Supabase Auth JWT) -> RLS tables + RPCs (`decide_drafts`, `trash_*`, ...). AI: client -> Edge Function `docs-mcp` -> `mcp_*` SQL functions (service role) -> `private.mcp_gate` (token, limits) -> drafts only. Accepting runs `private.apply_draft`. Realtime tells the owner's browser to refetch.

R0: planned restructure (NOT done; do it after the product model, with a green test run before and after)

The code is split by number prefix inside one flat `parts/`. Target:

```
src/app/ (shell, routing, store)   src/features/{reader,ai-
space,pages,trash,access,projects,orgs,editor}/   src/lib/{supabase,markdown,diff}/
supabase/{migrations,functions,seed}   tests/{e2e,mcp,content}
tools/{debug,profile,screenshots}   docs/
```

Steps: (1) keep `build.mjs` working by adding a manifest file listing parts in order; (2) move one feature at a time into src/features with real ES imports; (3) once every part is a module, delete build.mjs and the `canvas/` mechanism (use a Vite alias for the Canvas build); (4) move dbg/prof/shots to tools/. Why not now: the concatenated single-module design shares globals; renaming breaks the Canvas stubs and every test selector at once.

UPDATE 2026-10-21: see docs/14 (MCP 2026-07-28 spec, per-client matrix, Supabase OAuth beta caveats, token format, review friction). Where this file says UNVERIFIED for Supabase OAuth, docs/14 now has the verified facts.

Security and token safety

Built (verified)

Hashed tokens (sha256, never stored raw, shown once); modes read|propose; id-prefix limits; daily limits; pending cap; pause switch; per-address limits + failed-token lockout (0012); RLS everywhere; definer RPCs with `search_path=''`; hidden-Unicode and injection-phrase checks; tool output wrapped in `<untrusted-docs id=random>`; owner message via `get_guide`; audit log with error text; retention job; token secret immutable (column grants + trigger); AI can only create drafts.

Known weaknesses (open)

1. **Path tokens**: the Gemini app needs the token inside the URL. Supabase request logs store URLs (the owner saw full URLs in edge_logs; an audit counted hundreds of rows with tokens). Anyone with dashboard/log access can replay a token until it expires.
2. Tokens are long-lived bearer secrets with no binding to a client.
3. No alert when a token is used from a new client.
4. Leaked-password protection (Supabase Auth setting) is off.
5. `private.apply_draft`/`derived_edit` still granted to `authenticated` (not reachable via API; harden later).
6. Count-then-insert races on the pending cap and idempotency.

Researched direction (sources fetched 2026-10-20; secondary sources marked)

- MCP authorization spec 2026-07-28 security considerations (modelcontextprotocol.io/specification/2026-07-28/basic/authorization/security-considerations): clients send the RFC 8707 `resource` parameter, servers must reject tokens whose audience is not them, servers must not pass the client token upstream, tokens must not be logged/cached insecurely.
- Secondary (NeuralTrust, Oct 2026; Zuplo "bind MCP tokens to one server"; cenrax substack): send tokens in the Authorization header, never the query string; short-lived access tokens with refresh rotation; bind tokens to one server.
- UNVERIFIED: that the Gemini app supports OAuth for custom MCP servers (the owner only confirmed URL-path tokens work).

Upgrade plan (ordered, smallest first)

1. **Two token kinds**: `header` (long expiry allowed, read or propose) and `path` (read-only by default, max 7 days, low daily limit, labelled "appears in logs"). Enforce in `create_mcp_token` and `mcp_gate`.
2. **Rotation with overlap**: "Rotate" creates a new token and keeps the old one valid for N hours, then revokes it.
3. **Token prefix + checksum** (`gfk_live_...`) so GitHub secret scanning / gitleaks can detect leaks; show last 4 characters in the list.
4. **Anomaly alerts**: store `last_client`, first-seen client set, call spikes; show a warning chip and optionally auto-pause.
5. **Short-lived path tokens via one-click "Connect Gemini"**: owner creates a 24 h read token from the app, one tap.

6. **OAuth 2.1 for MCP** (Supabase Auth OAuth server was reported in beta by one source: UNVERIFIED) so clients that support it get short-lived audience-bound tokens; keep manual tokens as fallback.
7. Redact tokens from our own logs (we never log them; Supabase gateway does -> cannot fix, only mitigate with 1 and 5).
8. Turn on leaked-password protection; add TOTP MFA for the owner account; consider passkeys.
9. Revoke the two broad grants (item 5 above) after verifying `decide_drafts` and `revert_draft` still work (test with a pending draft in a rolled-back DO block).
10. Security tests to add: RLS matrix test (anon / non-member / viewer / editor / admin x every table) as a script that runs in CI.

API and integration plan

Rule: keys never go in chat, code or git. Storage: Supabase Edge Function secrets (dashboard dialog or `create_edge_function_secret` tool which opens a dialog). Browser never sees them; every call goes through an Edge Function that checks the signed-in user and rate-limits.

Priority	Integration	Use	Notes
1	**CHANGED 2026-10-21 (docs/15): Anthropic API, Claude Haiku 5.5 with Citations, prepaid with a ~5 USD monthly cap, key only as an Edge Function secret; Gemini paid tier as fallback. No OpenRouter key rotation** (terms 7(3) ban multiple accounts to bypass limits; extra keys do not raise limits; free tiers may train on prompts). Old row: OpenRouter with 5 keys, now dropped.	ask-the-docs, summaries, "why this changed", draft step text	verbatim quote check on every answer; see docs/15 for the algorithm, limits and cost guard
2	GitHub REST/GraphQL (fine-grained token, repo-scoped)	link phases/steps to issues & PRs, push Markdown export to a repo, status badges	read-only first
3	Supabase Storage	file attachments per project (PDF, images), signed URLs	RLS on storage.objects
4	Resend / SMTP	digest of pending AI edits, token-expiry mails	one Edge Function, opt-in
5	Web Push (VAPID, no third party)	"new AI edit waiting" notification	service worker
6	Vercel API	show deploy status in the app	read-only token
7	Cloudflare Turnstile	protect public forms/signup	
8	Embeddings (pgvector is available in the project)	semantic search	model via OpenRouter or Supabase AI inference (UNVERIFIED availability)
9	Calendar (ICS feed generated by an Edge Function)	phase due dates in any calendar app	no OAuth needed
10	Webhooks out (signed)	notify other systems when a decision is made	HMAC header
Research to do before building (not yet done): GitHub search for MCP gateways/admin UIs (Bifrost, Obot, Lunar MCPX, Microsoft MCP Gateway, mcphub, Unla were named by a research run, UNVERIFIED licences/dates), and documentation platforms (Docmost, AFFiNE, BookStack, Outline: spaces/nested pages/revision history/MCP servers exist per a search, details UNVERIFIED).			

Known limits and fixes

#	Limit	Fix
1	Only one project and one admin; spec chapters are generated (`derived`), edited only through AI inbox	docs/01 steps 1-6
2	Free plan: project pauses after ~1 week of inactivity (pg_cron then does not run)	purge also runs when Trash opens; upgrade or keep-alive ping
3	Realtime free plan: 200 connections, public visitors get no live updates (they check on focus)	fine for now
4	supabase-js WebSocket issues (2768, 2613): explicit WebSocket transport set; resync on SUBSCRIBED; retry loop fixed	monitor
5	Calls rejected by the Edge Function's zod validation leave no audit row	log in Edge Function before returning (small change in core.mjs)
6	No per-revision deletion, no rebase of stale AI edits	stale edit -> "ask AI to redo" button
7	Pending-cap / idempotency races	advisory lock on token id in mcp_propose_set
8	Trashed page with an applied AI edit is purged after the retention time	retention is becoming a per-project setting (below)
9	Peek panel on phones is not a true modal (no focus trap)	make it a `<dialog>` below 900 px
10	Test rows remain (revoked e2e tokens, rejected drafts)	delete them with Trash/Clear tools
11	`diffWordsWithSpace` on huge texts may be slow	jsdiff `timeout` option
12	Canvas single-file build lacks all cloud features	by design (stubs)
13	Large files: parts/ is one global scope	restructure R0
14	Edge Function must be redeployed by pasting files (deploy tool)	add `supabase functions deploy` to CI with a token

Trash timer (owner request)

Make retention a setting: `projects.settings.trash_days` (1-365, default 30) and a per-delete choice ("keep in Trash for 7 / 30 / 90 days / forever"). Implementation: `trash_edits/trash_pages` read the project setting instead of the literal `interval '30 days'`; Trash page shows "purges in N days" (exists) and lets the owner change the date of one item (`update ... set purge_after`) via a new RPC `set_purge_after(ids, ts)`. Cron and `purge_trash()` already honour `purge_after`.